

มหาวิทยาลัยราชภัฏบุรีรัมย์ — BRU Strategic Tracking System

คู่มืออธิบายสถาปัตยกรรมโค้ดและการทำงานทั้งระบบฉบับสมบูรณ์ (Codeeee.pdf)

Full-Stack Architecture

ฉบับละเอียด & สมบูรณ์

เอกสารอธิบายสถาปัตยกรรมโค้ดและการทำงานทั้งระบบฉบับสมบูรณ์

BRU Strategic Performance Tracking System — Full-Stack Deep Dive & Code Manual (Codeeee.pdf)

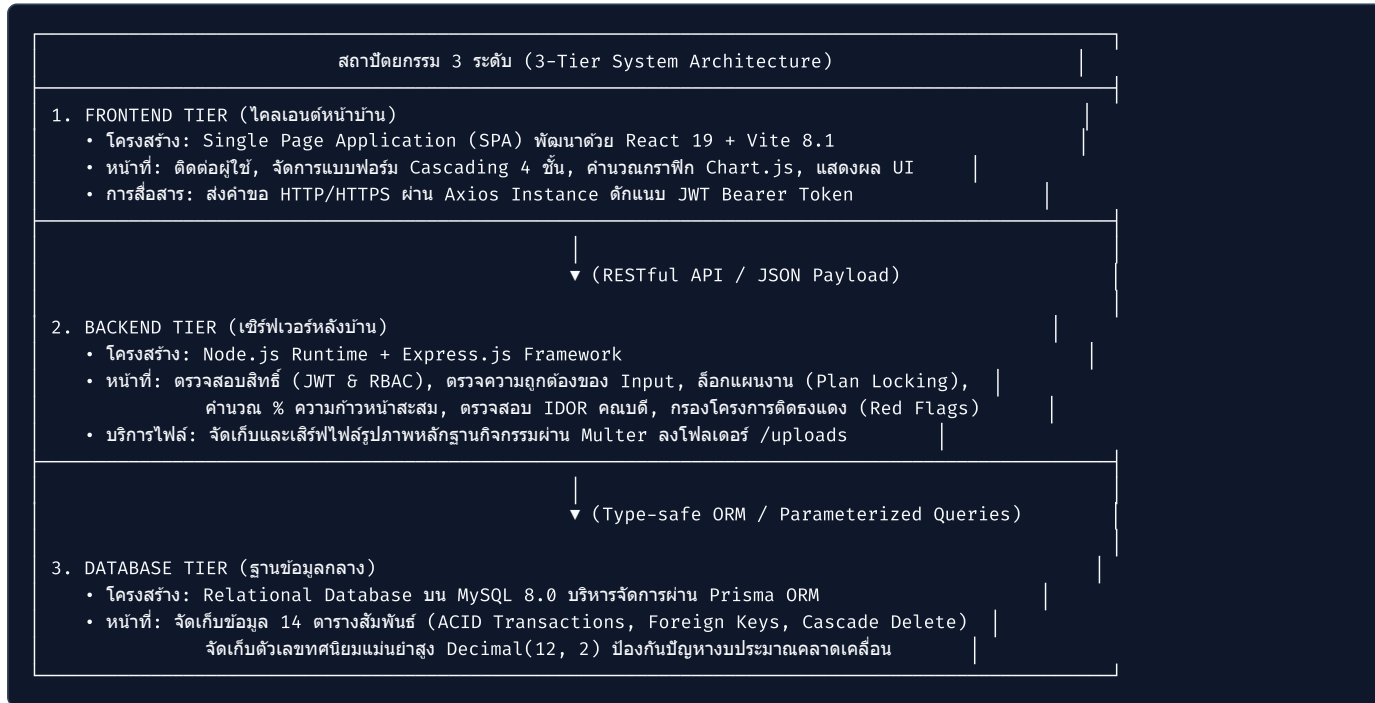
มหาวิทยาลัยราชภัฏบุรีรัมย์ | รอบปีงบประมาณ 2569

สารบัญหัวข้อหลัก

- บทนำและสถาปัตยกรรมภาพรวมของระบบ (Big Picture Architecture)
- ภาษาคอมพิวเตอร์ที่ใช้ในระบบ คืออะไรและทำอะไรได้บ้าง? (Languages Explained)
- เครื่องมือและเทคโนโลยีทั้งหมดในระบบ (Complete Tech Stack Breakdown)
- เจาะลึกฐานข้อมูล 14 ตาราง และข้อมูลระบบ (Database & Master Data)
- โครงสร้างโค้ดฝั่งหลังบ้าน (Backend Server: ทุกไฟล์ ทุกโฟลเดอร์)
- โครงสร้างโค้ดฝั่งหน้าบ้านและเลย์เอาต์หน้าเว็บ (Frontend UI: ทุกไฟล์ ทุกโฟลเดอร์)
- ที่อยู่ของสี ธีม ภาพ และไฟล์มัลติมีเดีย (Colors, Themes & Assets Location)
- การทำงานร่วมกันตั้งแต่หน้าบ้านถึงฐานข้อมูล (End-to-End Collaboration Pipeline)
- คู่มือการ Deploy ระบบสู่สภาพแวดล้อมจริงโดยละเอียด (Deployment Architecture)
- สารบัญสรุปแผนผังโฟลเดอร์และไฟล์ทั้งหมดในโปรเจกต์ (Full Inventory Tree)

1. บทนำและสถาปัตยกรรมภาพรวมของระบบ

ระบบติดตามและประเมินผลโครงการตามยุทธศาสตร์ มหาวิทยาลัยราชภัฏบุรีรัมย์ ถูกพัฒนาขึ้นด้วยสถาปัตยกรรม **3-Tier Architecture (Client - Server - Database)** แบบแยกส่วนอิสระ (Decoupled Micro-layers) เพื่อรองรับการทำงานแบบเรียลไทม์ ความปลอดภัยของข้อมูลตามบทบาท (RBAC) และการจัดการแบบข้อยกเว้น (Management by Exception):



2. ภาษาคอมพิวเตอร์ที่ใช้ในระบบ คืออะไรและทำอะไรได้บ้าง?

เพื่อให้เข้าใจว่าโค้ดแต่ละไฟล์เขียนด้วยภาษาอะไร และทำหน้าที่อะไรในระบบ นี่คือการอธิบายของทุกภาษาที่ใช้พัฒนา:

ภาษาที่ใช้	มันคือภาษาอะไร?	ทำหน้าที่อะไรในระบบนี้?	ตัวอย่างไฟล์ในระบบ
JavaScript (ES6+)	ภาษาโปรแกรมเชิงฟังก์ชันและเชิงวัตถุที่ทำงานได้ทั้งฝั่ง Browser และฝั่ง Server (ผ่าน Node.js)	• หลังบ้าน: ใช้ขับเคลื่อนเซิร์ฟเวอร์ Express, คำนวณงบประมาณ, ตรวจสอบสิทธิ์ JWT, กรอง Red Flags • หน้าบ้าน: จัดการตรรกะ Component, ดึง API, คำนวณผลผลิต	<code>backend/app.js</code> <code>frontend/src/services/api.js</code>
JSX (JavaScript XML)	ไวยากรณ์ส่วนขยายของ JavaScript ที่พัฒนาโดยทีม React ทำให้สามารถเขียนโครงสร้างหน้าตาคล้าย HTML ผสมกับโค้ด JavaScript ได้โดยตรง	ใช้สร้างคอมโพเนนต์หน้าจอ เช่น Topbar, Sidebar, ฟอร์มกรอกโครงการ, โมดัล และแดชบอร์ด โดย React จะแปลง JSX เป็น Virtual DOM เพื่อเรนเดอร์ลงเบราว์เซอร์อย่างรวดเร็ว	<code>frontend/src/layouts/AppLayout.jsx</code> <code>frontend/src/pages/teacher/ProjectForm.jsx</code>
HTML5	ภาษาโครงสร้างพื้นฐานสำหรับแสดงผลหน้าเว็บ (HyperText Markup Language)	เป็นโครงสร้างจุดเริ่มต้น (Root Document) ที่เตรียมแท็ก <code><div id="root"></div></code> ไว้เพื่อให้ React ทำการ Mount คอมโพเนนต์ทั้งหมดขึ้นแสดงบนหน้าจอ	<code>frontend/index.html</code>
CSS3 & Tailwind CSS	ภาษาจัดรูปแบบสไตล์หน้าเว็บ โดยระบบนี้ใช้ Tailwind CSS ซึ่งเป็น Utility-first CSS Framework	กำหนดสีพื้น, ฟอนต์, ขอบ, ระยะห่าง (Padding/Margin), เงา (Shadow), การจัดหน้าจอแบบ Flexbox/Grid และรองรับ Responsive บนมือถือ/แท็บเล็ตโดยไม่ต้องเขียนไฟล์ CSS แยกย่อยๆ	<code>frontend/tailwind.config.js</code> <code>frontend/src/index.css</code>

ภาษาที่ใช้	มันคือภาษาอะไร?	ทำหน้าที่อะไรในระบบนี้?	ตัวอย่างไฟล์ในระบบ
SQL (Structured Query Language)	ภาษามาตรฐานสำหรับสอบถามและจัดการข้อมูลในฐานข้อมูลเชิงสัมพันธ์ (Relational Database)	ใช้กำหนดโครงสร้างตาราง DDL (CREATE TABLE), กำหนด Foreign Key, และคำสั่ง INSERT INTO ข้อมูลเริ่มต้น (Seed) บน MySQL	database/schema.sql database/seed.sql
Prisma Schema Language (PSL)	ภาษาเฉพาะ (Declarative Modeling Language) ของ Prisma ORM	ใช้ประกาศโมเดลโครงสร้างตาราง 14 ตาราง, ความสัมพันธ์ 1:N และ M:N, กำหนด Enums สิทธิ์ และถูกคอมไพล์เป็น TypeScript/JavaScript Client เพื่อให้เรียกใช้ฐานข้อมูลแบบ Type-safe	backend/prisma/schema.prisma
JSON (JavaScript Object Notation)	รูปแบบมาตรฐานเปิดสำหรับแลกเปลี่ยนข้อมูลแบบข้อความที่มีโครงสร้าง Key-Value	- ใช้เป็นรูปแบบข้อมูลกลางที่ส่งไปมาระหว่าง Frontend และ Backend - ใช้เก็บการตั้งค่าโครงการ (package.json) - ใช้เป็นรูปแบบของชุดทดสอบ Postman Collection	backend/package.json postman/BRU_Strategic_Tracking_API_Test_Collection.postman_collection.json
Markdown (.md)	ภาษาจัดรูปแบบข้อความอย่างง่ายสำหรับเขียนเอกสาร	ใช้บันทึกพจนานุกรมข้อมูล, คู่มือการนำเสนอ, และโครงสร้างระบบ	present.md database/DataDictionary.md

3. เครื่องมือและเทคโนโลยีทั้งหมดในระบบ (Complete Tech Stack Breakdown)

เครื่องมือและเทคโนโลยีที่ใช้พัฒนาระบบ		
หมวดหมู่	เครื่องมือที่เลือกใช้	บทบาทและเหตุผลทางวิศวกรรมซอฟต์แวร์
1. Frontend UI Core	React 19.0.0	Virtual DOM เรนเดอร์เร็ว, สถาปัตยกรรม SPA
2. Frontend Bundler	Vite 8.1.4	เครื่องมือ Build ความเร็วสูง โหลดเร็ว 3วิ
3. CSS & Styling	Tailwind CSS + PostCSS	Utility Classes ยืดหยุ่น คมสีสถาบันแม่นยำ
4. Navigation/Routing	React Router DOM v6	จัดการเปลี่ยนหน้าแบบ Client-side ไม่รีเฟรช
5. Data Visualization	Chart.js + React-Chartjs-2	แสดงกราฟแท่ง งบประมาณ และ Strategic Chart
6. Icons	React Icons (Feather Icons)	ไอคอน Vector SVG คมชัด ไม่แตก ปรับสีได้
7. Notifications	SweetAlert2	กล่องข้อความแจ้งเตือน Pop-up หรรษา สวยงาม
8. Form Management	React Hook Form	จัดการฟอร์มขนาดใหญ่ ไม่ Re-render พრაเพอร์
9. Backend Runtime	Node.js (v18+)	Non-blocking I/O รองรับ Request พร้อมกันสูง
10. Web Framework	Express.js (v4.19)	Middleware Architecture แข็งแกร่ง เป็นสากล
11. Database Engine	MySQL 8.0 Community	ฐานข้อมูลเชิงสัมพันธ์ ทนทาน รองรับ ACID
12. ORM & Migration	Prisma ORM (v5.12)	Type-safe Queries, ป้องกัน SQL Injection
13. Security & Hash	Bcrypt.js (10 Salt Rounds)	แฮชรหัสผ่านทางเดียวแบบ One-way Hash
14. Authentication	JSON Web Token (JWT)	Stateless Authentication มาตรฐานสากล
15. File Uploading	Multer (v1.4)	รับไฟล์ภาพหลักฐานกิจกรรม ตรวจสอบขนาด 5MB
16. HTTP Client	Axios (v1.6)	ทำ Interceptor แบน Bearer Token จัดโน้มนัด
17. Security Headers	Helmet.js	ตั้งค่าความปลอดภัย HTTP Headers ป้องกัน XSS
18. API Test Suite	Postman Collection v2.1	ชุดทดสอบ API จัดโน้มนัด 8 หมวด 25+ Requests

4. เจาะลึกฐานข้อมูล 14 ตาราง และข้อมูลระบบ (Database & Master Data)

โครงสร้างฐานข้อมูลถูกประกาศไว้ที่ไฟล์ `backend/prisma/schema.prisma` มีตารางทั้งหมด **14 ตาราง** แบ่งออกเป็น 5 กลุ่มงานอย่างเป็นระบบ:

โครงสร้าง 14 ตารางฐานข้อมูล	
[กลุ่มที่ 1: ผู้ใช้งานและสิทธิ์ (3 ตาราง)]	
• users	: เก็บบัญชีผู้ใช้, รหัสผ่านแฮช Bcrypt, สิทธิ์ Role (TEACHER, DEAN, PRESIDENT, ADMIN)
• faculties	: ข้อมูลคณะทั้ง 9 คณะของมหาวิทยาลัย
• departments	: ข้อมูล 61 ภาควิชา/สาขาวิชาที่สังกัดอยู่แต่ละคณะ (FK → faculties.id)
[กลุ่มที่ 2: โครงสร้างยุทธศาสตร์ 4 ระดับ (Cascading Strategy 4 ตาราง)]	
• local_issues	: ยุทธศาสตร์ระดับ 1 – ประเด็นการพัฒนาท้องถิ่น (Local Development Issues - LDI)
• strategies	: ยุทธศาสตร์ระดับ 2 – แผนงานหลัก 6 เสาหลัก (Strategic Pillars S1-S6)
• sub_strategies	: ยุทธศาสตร์ระดับ 3 – แผนงานย่อย (FK → strategies.id)
• indicators	: ยุทธศาสตร์ระดับ 4 – โครงการหลัก 10 โครงการ (Main Projects - MP) (FK → sub_strategies)
[กลุ่มที่ 3: โครงการและการบริหาร (3 ตาราง)]	
• projects	: ข้อมูลโครงการ, งบประมาณ, เป้าหมาย, วันที่เริ่ม-สิ้นสุด, สถานะล็อก (isLocked), ข้อสั่งการคณะบดี (deanDirective), ข้อสั่งการอธิการบดี (presidentDirective)
• project_users	: ตาราง Many-to-Many เชื่อมโยงโครงการกับอาจารย์ผู้ร่วมรับผิดชอบหลายท่าน
• fiscal_years	: ปีงบประมาณ (เช่น 2569)
• budget_sources	: แหล่งงบประมาณ (งบแผ่นดิน, งบรายได้, งบวิจัย)
[กลุ่มที่ 4: กิจกรรมและภาพถ่ายหลักฐาน (2 ตาราง)]	
• activities	: กิจกรรมย่อยใดโครงการ, วันที่ดำเนินงาน, งบประมาณใช้จริง, ผลผลิตที่ทำได้จริง
• activity_images	: เก็บ Path สัมพันธ์ของรูปภาพหลักฐานกิจกรรมที่อัปโหลด (FK → activities.id)
[กลุ่มที่ 5: ระบบบริการและแจ้งปัญหา (1 ตาราง)]	
• issues	: รายการแจ้งปัญหาระบบของผู้ใช้ส่งถึงผู้ดูแลระบบ (Status: PENDING, RESOLVED)

ข้อมูลระบบ (Master Data) ทั้ง 9 หมวด

ข้อมูลระบบในหน้าระบบจัดการข้อมูลของผู้ดูแลระบบ (`pages/admin/MasterData.jsx`) ประกอบด้วย:

- บัญชีผู้ใช้งาน (Users): จัดการเพิ่ม/ระงับบัญชี และกำหนด Role
- ประเด็นการพัฒนาท้องถิ่น (Local Issues): LDI ระดับที่ 1
- แผนงานหลัก (Strategies): ยุทธศาสตร์ระดับที่ 2
- แผนงานย่อย (Sub-strategies): ยุทธศาสตร์ระดับที่ 3
- โครงการหลัก (Indicators / Main Projects): ยุทธศาสตร์ระดับที่ 4
- คณะ (Faculties): 9 คณะ
- ภาควิชา/หน่วยงาน (Departments): 61 สาขาวิชา
- ปีงบประมาณ (Fiscal Years): 2569
- แหล่งเงินงบประมาณ (Budget Sources): งบแผ่นดิน / งบรายได้ / งบวิจัย

5. โครงสร้างโค้ดฝั่งหลังบ้าน (Backend Server: ทุกไฟล์ ทุกโฟลเดอร์)

โฟลเดอร์: `c:\St_bru\backend\`

5.1 ไฟล์หลักระดับราก (Root Files)

- **app.js** : จุดเริ่มต้นของเซิร์ฟเวอร์ Express
 - เปิดพอร์ต 5000 รันเซิร์ฟเวอร์
 - โหลด Middleware ความปลอดภัย: `helmet` (Headers Security), `cors` (อนุญาตการเรียกข้ามโดเมน), `rate-limit` (ป้องกัน Brute Force)
 - ตั้งค่าบริการไฟล์รูปภาพ: `app.use('/uploads', express.static(...))` ให้นำหน้าเว็บเปิดดูรูปได้
 - เชื่อมต่อ Route ทั้งหมด 8 หมวดหมู่เข้าสู่ Prefix `/api`
- **package.json** : ระบุไลบรารี Dependencies และสคริปต์รัน เช่น `npm start` (`node app.js`)
- **.env** : จัดเก็บ Environment Variables ที่มีความลับ เช่น:
 - `DATABASE_URL="mysql://root:password@localhost:3306/bru_strategic_tracking"`
 - `JWT_SECRET="supersecretjwtkey ..."`
 - `PORT=5000`
- **verify-endpoints.js** : สคริปต์ตรวจสอบความสมบูรณ์และทดสอบการเชื่อมต่อ API ของระบบ 5 ด้าน
- **seed-projects.js** : สคริปต์สร้างข้อมูลโครงการและกิจกรรมจำลองสำหรับการทดสอบ

5.2 โฟลเดอร์ `config/` (การตั้งค่าระบบ)

- **config/prisma.js** : ตัวสร้าง Singleton Instance ของ PrismaClient
 - ทำไมต้องทำเป็น Singleton? เพื่อป้องกันไม่ให้ Node.js สร้าง Connection ไปยังฐานข้อมูล MySQL ซ้ำซ้อนในทุก Request ซึ่งอาจทำให้ Connection ของ MySQL เต็มและระบบค้าง
- **config/cloudinary.js** : ตัวเชื่อมต่อบริการจัดเก็บรูปภาพ Cloudinary บนระบบคลาวด์ (ตัวเลือกเสริม)

5.3 โฟลเดอร์ `middleware/` (ตัวคัดกรองคำขอก่อนเข้า Controller)

- **middleware/auth.middleware.js** :
 - `authenticate` : ตรวจสอบความถูกต้องของ Header `Authorization: Bearer <token>` ถอดรหัส JWT และนำข้อมูล User ไปฝากไว้ที่ `req.user`
 - `authorize([...roles])` : ตรวจสอบสิทธิ์ เช่น อนุญาตเฉพาะ `['ADMIN']` หรือ `['DEAN', 'PRESIDENT']` หากไม่มีสิทธิ์จะส่ง `HTTP 403 Forbidden`
- **middleware/validation.middleware.js** :
 - ใช้ `express-validator` ตรวจสอบความถูกต้องของข้อมูล Input ป้องกันข้อมูลขยะหรือตัวเลขติดลบ เช่น ตรวจสอบงบประมาณห้ามติดลบ, รูปแบบวันที่ต้องเป็น ISO8601
- **middleware/upload.middleware.js** :
 - ใช้ไลบรารี **Multer** ในการดักจับไฟล์รูปภาพ ตรวจสอบว่าต้องเป็นไฟล์นามสกุลภาพเท่านั้น (`jpeg`, `jpg`, `png`, `webp`) และจำกัดขนาดไฟล์ไม่เกิน 5 MB ต่อรูป พร้อมเปลี่ยนชื่อไฟล์ใหม่โดยใส่ Timestamp ป้องกันชื่อไฟล์ซ้ำกัน

5.4 โฟลเดอร์ `routes/` (เส้นทาง API)

- **auth.routes.js** : เส้นทางเข้าสู่ระบบ (`/login`), ดึงโปรไฟล์ปัจจุบัน (`/me`), เปลี่ยนรหัสผ่าน, และสถิติสาธารณะ
- **project.routes.js** : เส้นทางเพิ่ม, แก้ไข, ลบ, ค้นหาโครงการ และสลับ/ปลดล็อกแผนงาน (`/toggle-lock`)
- **activity.routes.js** : เส้นทางเพิ่มกิจกรรมย่อย, บันทึกผลการดำเนินงาน (`/progress`), อัปโหลดภาพหลักฐาน, และลบกิจกรรม

- `directive.routes.js` : เส้นทางสำหรับบันทึกข้อสั่งการเร่งรัดงานของผู้บริหาร (`/dean` , `/president`)
- `dashboard.routes.js` : เส้นทางดึงข้อมูลสรุปแดชบอร์ดตาม Role (`/` , `/dean` , `/president`)
- `master.routes.js` : เส้นทางจัดการข้อมูล Master Data 9 หมวด (ยุทธศาสตร์, คณะ, สาขา, แหล่งเงิน)
- `issue.routes.js` : เส้นทางส่งเรื่องแจ้งปัญหาของผู้ใช้งาน และการอัปเดตสถานะของแอดมิน
- `report.routes.js` : เส้นทางรวบรวมข้อมูลสรุปโครงการ และส่งออกไฟล์ CSV / Excel / PDF

5.5 ไวลเดอร์ `controllers/` (ถสกะทางธุรกิจ Business Logic)

- `project.controller.js` :
 - ระบบล็อกแผนงาน (Plan Locking): ตรวจสอบว่าโครงการมีกิจกรรมย่อยหรือยัง หากมีแล้วและผู้ใช้ไม่ใช่อาจารย์แอดมิน จะไม่อนุญาตให้แก้ไขงบประมาณรวมและเป้าหมาย เพื่อรักษาธรรมชาติ
 - ระบบป้องกัน IDOR: ในฟังก์ชัน `updateExecutiveDirective` ตรวจสอบว่าคณบดีที่สั่งการสังกัดคณะเดียวกับโครงการหรือไม่ หากไม่ใช่จะปฏิเสธด้วย `HTTP 403`
 - การคำนวณ % ความก้าวหน้าสะสม: รวมผลผลิตจากกิจกรรมที่สถานะเสร็จสิ้น (`COMPLETED`) แล้วหารด้วยเป้าหมายโครงการ (จำกัดสูงสุด 100%)
- `dashboard.controller.js` :
 - ระบบคัดกรองโครงการติดธงแดง (MBE Red Flags): ตรวจสอบ 2 เงื่อนไข (เลยกำหนดวันสิ้นสุดแต่ยังไม่เสร็จ 100%, หรือเวลาผ่านไปเกิน 50% แต่งานทำได้ไม่ถึง 25%) หากเข้าเงื่อนไขจะถูกติดธงแดงขึ้นแดชบอร์ดผู้บริหาร
- `activity.controller.js` : จัดการบันทึกกิจกรรมย่อย นำรูปภาพที่ Multer อัปโหลดเข้าสู่ตาราง `activity_images` และคำนวณยอดเงินเบิกจ่ายสะสม
- `auth.controller.js` : ตรวจสอบรหัสผ่านด้วย `bcrypt.compare()` , สร้าง JWT Token ด้วย `jwt.sign()` ที่มีอายุใช้งานและระบุ Role
- `master.controller.js` : จัดการ CRUD ข้อมูลยุทธศาสตร์ คณะ สาขา แหล่งเงินทุน และปีงบประมาณ
- `report.controller.js` : ประมวลผลสถิติงบประมาณและผลสัมฤทธิ์ แปลงเป็นไฟล์ CSV (รองรับภาษาไทยด้วย UTF-8 BOM) และไฟล์ Excel

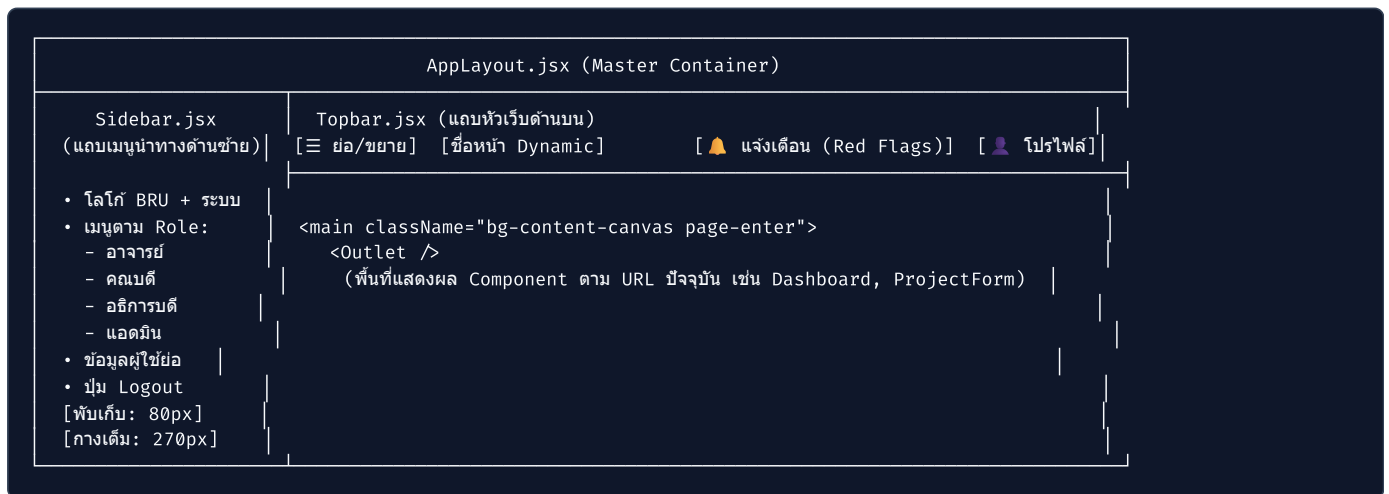
6. โครงสร้างโค้ดฝั่งหน้าบ้านและเลย์เอาต์หน้าเว็บ (Frontend UI: ทุกไฟล์ ทุกโฟลเดอร์)

โฟลเดอร์: `c:\St_bru\frontend\`

6.1 จุดเริ่มต้นของแอปพลิเคชัน

- `index.html` : โครงสร้าง HTML หลัก จุดติดตั้ง `<div id="root"></div>`
- `src/main.jsx` : จุดเริ่มต้น React นำ `<App />` ไปเรนเดอร์ใน DOM พร้อมห่อหุ้มด้วย `BrowserRouter`
- `src/App.jsx` : ศูนย์กลางระบบ Routing และ Role Guard
 - กำหนดเส้นทาง URL ทั้งหมดของระบบ
 - ใช้ **Role-based Protection**: ป้องกันไม่ให้อาจารย์เข้าหน้าของอธิการบดี หรือป้องกันไม่ให้ผู้ใช้ที่ไม่ได้ล็อกอินเข้าถึงแดชบอร์ด

6.2 สถาปัตยกรรมโครงสร้างหน้าเว็บ (Master Layout: `src/layouts/AppLayout.jsx`)



- `src/layouts/AppLayout.jsx` :
 - ควบคุม State การย่อ/ขยายเมนูด้านซ้าย (`sidebarCollapsed: 80px / 270px`)
 - ควบคุมการเปิด-ปิดเมนูบนหน้าจอมือถือ (`sidebarOpen`)
 - จัดการโหมดสแตนด์บาย (`Ctrl + P`) โดยซ่อนเมนูอัตโนมัติ
- `src/components/Sidebar.jsx` :
 - แสดงตราสัญลักษณ์มหาวิทยาลัยราชภัฏบุรีรัมย์ และชื่อระบบ
 - ไล่เฉดสีม่วงเข้มอันเป็นเอกลักษณ์ (`linear-gradient(180deg, #2F1481 0%, #1E0A4A 100%)`)
 - กรองแสดงเมนูอัตโนมัติตามสิทธิ์ของผู้ใช้ (`user.role`)
 - มีปุ่มลูกศรพับเก็บเมนูเหลือ 80px แสดงเฉพาะไอคอน เพื่อเพิ่มพื้นที่ทำงาน
- `src/components/Topbar.jsx` :
 - Dynamic Page Title**: คำนวณชื่อหน้าปัจจุบันอัตโนมัติตาม URL และ Role
 - Notification Center (กระดิ่ง FiBe11)**: ดึงโครงการติดธงแดงมาแจ้งเตือน มีตัวเลข Badge สีแดง พร้อมระบบบันทึกสถานะอ่านแล้วลง `localStorage`
 - User Profile Menu**: เมนูจัดการโปรไฟล์, เปลี่ยนรหัสผ่าน, แจ้งปัญหาระบบ, และปุ่มออกจากระบบ
- `<Outlet />` ใน `AppLayout.jsx` :
 - พื้นที่ตรงกลางสำหรับแสดงผล Component หน้าจอตามเส้นทาง URL
 - มีคลาส `bg-content-canvas` (`#F8FAFC`) สีพื้นหลังนุ่มนวลสบายตา และเอฟเฟกต์ Fade-in นุ่มนวล

6.3 คอมโพเนนต์ส่วนกลางและโมดัล (`src/components/`)

- `ProfileModal.jsx` : หน้าต่างป๊อปอัพข้อมูลส่วนตัว แก้ไขโปรไฟล์ และอัปโหลดภาพประจำตัว
- `ReportIssueModal.jsx` : หน้าต่างป๊อปอัพส่งเรื่องแจ้งปัญหาการใช้งานตรงถึงแอดมิน
- `ExecutiveProjectModal.jsx` : หน้าต่างป๊อปอัพสำหรับผู้บริหารดูรายละเอียดโครงการและบันทึกข้อสั่งการ
- `CustomSelect.jsx` : Dropdown พิเศษที่รองรับการพิมพ์ค้นหาตัวเลือก (Searchable Dropdown) สำหรับเลือกยุทธศาสตร์
- `ErrorBoundary.jsx` : ตัวดักจับข้อผิดพลาดของ React เพื่อป้องกันไม่ให้น้ำเว็บล่มทั้งจอเมื่อเกิด Bug

6.4 การจัดการสแตตและบริกาารเชื่อมต่อ (`src/contexts/`, `src/services/`, `src/utils/`)

- `contexts/AuthContext.jsx` : ศูนย์กลางจัดการ State การล็อกอิน เก็บข้อมูลผู้ใช้และ JWT Token พร้อมฟังก์ชัน `login()` และ `logout()`
- `services/api.js` : Axios Instance กลาง มี **Request Interceptor** คอยดักแนบ Header `Authorization: Bearer <token>` ไปกับทุกคำขอโดยอัตโนมัติ
- `utils/imageUrl.js` : แปลง Path รูปภาพสัมพัทธ์จาก Server ให้กลายเป็น Full URL และฟังก์ชันบีบอัดรูปภาพก่อนอัปโหลด
- `utils/statusHelper.js` : ฟังก์ชันแปลงรหัสสถานะโครงการ/กิจกรรมเป็นข้อความภาษาไทยและสี Badge ที่ถูกต้อง

6.5 หน้าจอแยกตามบทบาทผู้ใช้งาน (`src/pages/`)

- `pages/auth/Login.jsx` : หน้าจอล็อกอินเข้าสู่ระบบ พร้อมสไลด์ภาพและปุ่มกรอกบัญชีเดโม 4 บทบาท
- `pages/teacher/` (อาจารย์):
 - `Dashboard.jsx` : สรุปโครงการที่ตนเองดูแล
 - `Projects.jsx` : หน้ารายการโครงการ
 - `ProjectForm.jsx` : ฟอर्मสร้าง/แก้ไขโครงการ พร้อม **Cascading Dropdown** ยุทธศาสตร์ 4 ระดับ
 - `ProjectDetails.jsx` : รายละเอียดโครงการ, บันทึกกิจกรรม, และสถานะล็อกแผนงาน
 - `Gallery.jsx` : แกลเลอรีรูปภาพหลักฐานการลงพื้นที่จริง
- `pages/dean/` (คณบดี):
 - `Dashboard.jsx` : แดชบอร์ดสรุปผลงานระดับคณะ, ตารางเปรียบเทียบสาขาวิชา, และปุ่มสั่งการ
 - `Projects.jsx` : รายการโครงการเฉพาะภายในคณะ
 - `Reports.jsx` : รายงานสรุปผลงานระดับคณะ
- `pages/president/` (อธิการบดี):
 - `Dashboard.jsx` : แดชบอร์ดรวมมหาวิทยาลัย, กราฟ 6 เสายุทธศาสตร์, Strategic Heatmap, และรายการธงแดง
 - `Projects.jsx` : รายการโครงการทั้งหมดทุกคณะในมหาวิทยาลัย
 - `Reports.jsx` : รายงานสรุปยุทธศาสตร์ระดับสถาบัน
- `pages/admin/` (ผู้ดูแลระบบ):
 - `MasterData.jsx` : หน้าจัดการข้อมูลระบบ 9 หมวด
 - `Issues.jsx` : หน้าระบบจัดการเรื่องร้องเรียน/แจ้งปัญหาการใช้งาน
 - `Projects.jsx` : บริหารจัดการและสั่งปลดล็อกโครงการ
- `pages/executive/ProjectDetail.jsx` : หน้ารายละเอียดโครงการแบบสรุปสำหรับผู้บริหาร

7. ที่อยู่ของสี ธีม ภาพ และไฟล์มัลติมีเดีย (Colors, Themes & Assets Location)

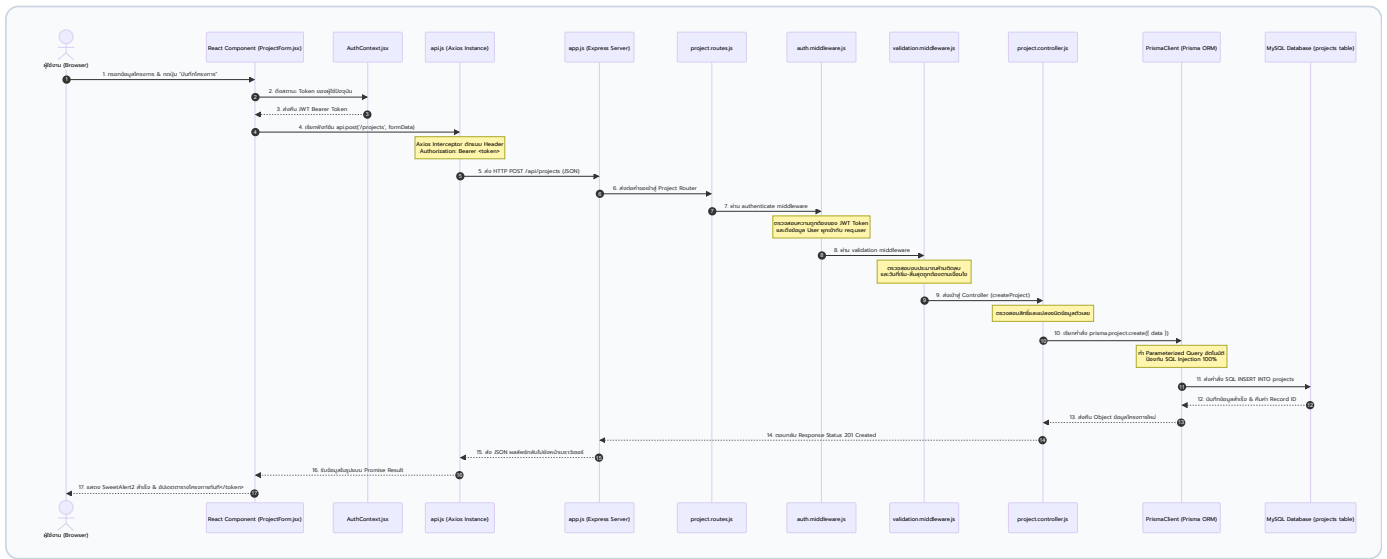
7.1 ตำแหน่งจัดเก็บและการตั้งค่าสีและธีม (Colors & Theme Palette)

ส่วนที่ต้องการเปลี่ยน	ไฟล์ที่จัดเก็บ	คำสั่งและค่าสีที่ใช้
สีหลักทั้งระบบ (Primary Theme)	frontend/tailwind.config.js	theme.extend.colors.primary.DEFAULT: '#6C3BFF' (สีม่วงเอกลักษณ์สถาบัน)
สีพื้นหลัง Canvas หน้าเว็บ	frontend/tailwind.config.js	theme.extend.colors['content-canvas']: '#F8FAFC' (สีเทาอ่อนนุ่มนวลสบายตา)
สีพื้นหลังเมนู Sidebar	frontend/src/components/Sidebar.jsx	linear-gradient(180deg, #2F1481 0%, #1E0A4A 100%) (สีม่วงเข้มไล่เฉด)
สีป้ายสถานะโครงการ (Badges)	frontend/src/utls/statusHelper.js	- เสร็จสิ้น: สีเขียว (bg-emerald-100 text-emerald-700) - กำลังดำเนินงาน: สีเหลือง/น้ำเงิน (bg-amber-100 text-amber-700) - ล่าช้า/ติดปัญหา: สีแดง (bg-rose-100 text-rose-700)
ฟอนต์และสไตล์สากล	frontend/src/index.css	เรียกใช้ Google Fonts (Prompt , Sarabun) และตั้งค่า Base CSS

7.2 ตำแหน่งจัดเก็บรูปภาพและไฟล์มัลติมีเดีย (Images & Assets)

```
c:\St_bru\  
├── frontend/public/ → [รูปภาพประจำระบบ / โลโก้สถาบัน / ภาพพื้นหลัง]  
│   ├── logob.png      - ตราสัญลักษณ์มหาวิทยาลัยราชภัฏบุรีรัมย์ (แสดงบน Topbar และ Sidebar)  
│   ├── icon.png / favicon.svg - ไอคอนสัญลักษณ์แสดงบนแท็บของเบราว์เซอร์  
│   ├── login1.jpg      - ภาพสไลด์พื้นหลังหน้าเข้าสู่ระบบ รูปที่ 1  
│   ├── login2.jpg      - ภาพสไลด์พื้นหลังหน้าเข้าสู่ระบบ รูปที่ 2  
│   └── login3.jpg      - ภาพสไลด์พื้นหลังหน้าเข้าสู่ระบบ รูปที่ 3  
├── backend/uploads/ → [โฟลเดอร์เก็บไฟล์ภาพกิจกรรมจริงที่อัปโหลดขึ้นเซิร์ฟเวอร์]  
│   └── (ไฟล์ภาพกิจกรรมจริง .jpg, .png ที่ถูกเปลี่ยนชื่อเป็น Timestamp เช่น 1787810803631.jpg)  
└── frontend/src/utls/imageUrl.js → [ฟังก์ชันจัดการแปลง URL และมีบีบอัดภาพ]  
    ├── getImageUrl(path) - เพิ่ม Prefix URL (เช่น http://localhost:5000) ให้ภาพจาก backend แสดงผลได้  
    └── compressImage(file) - บีบอัดขนาดภาพบนหน้า Browser ให้เล็กลงก่อนส่งขึ้น Server
```

8. การทำงานร่วมกันตั้งแต่หน้าบ้านถึงฐานข้อมูล (End-to-End Collaboration Pipeline)



9. คู่มือการ Deploy ระบบสู่สภาพแวดล้อมจริงโดยละเอียด (Deployment Architecture)

เมื่อต้องการนำระบบขึ้นสู่ Cloud หรือ Server จริงเพื่อใช้งานจริงในระดับ Production ระบบถูกออกแบบให้รองรับการ Deploy แยกส่วนได้อย่างง่ายดาย:

การ Deploy ระบบสู่ระบบคลาวด์ (Cloud Architecture)	
1. FRONTEND DEPLOYMENT (Vercel หรือ Cloudflare Pages)	
- เครื่องมือที่ใช้: Vercel CLI หรือ Git Integration บน Vercel - Build Command: npm run build - Output Directory: dist/ - ไฟล์สำคัญ: vercel.json (มีคำสั่ง Rewrite ทุก Request ไปที่ /index.html บล็อกหน้า 404 เมื่อรีเฟรช) - Environment Variable: VITE_API_URL=https://api.yourdomain.com/api	
2. BACKEND DEPLOYMENT (Render / Railway หรือ Ubuntu Cloud VPS)	
- เครื่องมือที่ใช้: Node.js v18+ พร้อม Process Manager (PM2) เพื่อให้เซิร์ฟเวอร์รันตลอด 24 ชม. - คำสั่งรันเบื้องหลัง: pm2 start app.js --name "bru-backend" - Web Server Reverse Proxy: Nginx ทำหน้าที่รับ Request พอร์ต 80/443 และส่งต่อมายังพอร์ต 5000 - ความปลอดภัย: ติดตั้ง SSL Certificate ฟรีด้วย Certbot (Let's Encrypt HTTPS) - ไฟล์และโฟลเดอร์สำคัญ: โฟลเดอร์ /uploads ต้องตั้งสิทธิ์ chown ให้อัปโหลดภาพได้	
3. DATABASE DEPLOYMENT (Cloud Managed MySQL)	
- ผู้ให้บริการแนะนำ: Aiven MySQL, PlanetScale, Railway, หรือ MySQL 8.0 บนเครื่อง VPS - การ Migrate ฐานข้อมูล: สั่ง npx prisma migrate deploy เพื่อสร้าง 14 ตารางบนเซิร์ฟเวอร์จริง - การใส่ข้อมูลเริ่มต้น: สั่ง node prisma/seed.js เพื่อนำเข้า Master Data ทั้ง 9 หมวด	

รายการตัวแปรสภาพแวดล้อม (Environment Variables) ที่ต้องตั้งค่าบน Cloud:

```
# การเชื่อมต่อฐานข้อมูล MySQL
DATABASE_URL="mysql://username:password@host:3306/bru_strategic_tracking?sslmode=require"

# รหัสลับสำหรับเข้ารหัส JWT Token (ควรตั้งให้ยาวและปลอดภัย)
JWT_SECRET="BRU_STRATEGIC_SUPER_SECRET_KEY_2026_PRODUCTION"

# พอร์ตสำหรับรัน Backend Server
PORT=5000

# การตั้งค่า CORS Domain ฟังก์ชันบ้าน
FRONTEND_URL="https://strategic-tracking.bru.ac.th"
```

10. สารบัญสรุปแผนผังโฟลเดอร์และไฟล์ทั้งหมดในโปรเจกต์ (Full Inventory Tree)

c:\St_bru\	
database/	(สคริปต์ DDL, Data Dictionary และ ERD)
DataDictionary.md	- พจนานุกรมข้อมูล อธิบายโครงสร้างฟิลด์และชนิดข้อมูลของ 14 ตาราง
ERD.md	- แผนภาพความสัมพันธ์เอนทิตี (ER Diagram) ของฐานข้อมูล
schema.sql	- สคริปต์ DDL ภาษา SQL สำหรับสร้างตารางบน MySQL
seed.sql	- สคริปต์ INSERT INTO ภาษา SQL ใส่ข้อมูลเริ่มต้น
postman/	(ชุดทดสอบ API สำหรับโปรแกรม Postman)
BRU_Strategic_Tracking_API_Test_Collection.postman_collection.json	- ชุดทดสอบ API 8 หมวดหมู่ (25+ Requests)
BRU_Strategic_Tracking_Environment.postman_environment.json	- ตัวแปร Environment ทดสอบระบบ
README.md	- คู่มือการ Import และทดสอบ API ใน 2 คลิ
backend/	(ระบบเซิร์ฟเวอร์ Node.js + Express + Prisma ORM)
.env	- ตัวแปรสภาพแวดล้อมระบบ (DATABASE_URL, JWT_SECRET, PORT)
app.js	- จุดเริ่มต้นเซิร์ฟเวอร์ Express, รวม Middleware และเชื่อมต่อ Route หลัก
package.json / package-lock.json	- กำหนด Dependencies และคำสั่งสคริปต์รันของ Backend
seed-projects.js	- สคริปต์สร้างข้อมูลโครงการและกิจกรรมจำลองสำหรับทดสอบ
verify-endpoints.js	- สคริปต์ทดสอบยิง API ตรวจสอบสถานะการทำงานของ Endpoint ต่างๆ
config/	
prisma.js	- Singleton Instance ของ PrismaClient ป้องกัน Connection ซ้ำซ้อน
cloudinary.js	- ตั้งค่าเชื่อมต่อบริการจัดเก็บรูปภาพ Cloudinary (ตัวเลือกเสริม)
middleware/	
auth.middleware.js	- ตรวจสอบ JWT Token (authenticate) และตรวจสอบสิทธิ์ตาม Role (authorize)
validation.middleware.js	- ตรวจสอบความถูกต้องของข้อมูล Input ป้องกันข้อมูลผิดรูปแบบ
upload.middleware.js	- ตัวจัดการ Multer ตรวจสอบนามสกุลไฟล์ภาพและจำกัดขนาดไม่เกิน 5MB
routes/	
auth.routes.js	- เส้นทาง API ล็อกอิน, ตรวจสอบผู้ใช้ปัจจุบัน, และเปลี่ยนรหัสผ่าน
project.routes.js	- เส้นทาง API เพิ่ม ลบ แก้ไข ล็อกแผนงาน และค้นหาโครงการ
activity.routes.js	- เส้นทาง API เพิ่มกิจกรรมย่อย อัปโหลดรูปภาพ และคำนวณผลผลิต
dashboard.routes.js	- เส้นทาง API ดึงข้อมูลสรุปแดชบอร์ดตามบทบาทผู้ใช้งาน (Role)
directive.routes.js	- เส้นทาง API ส่งข้อสั่งการเร่งรัดงานของผู้บริหาร (Dean / President)
master.routes.js	- เส้นทาง API จัดการข้อมูลพื้นฐานระบบ 9 หมวด (ยุทธศาสตร์, คณะ, สาขา)
issue.routes.js	- เส้นทาง API ส่งเรื่องแจ้งปัญหาการใช้งานและอัปเดตสถานะปัญหา
report.routes.js	- เส้นทาง API รวบรวมข้อมูลสรุปโครงการเพื่อจัดทำรายงาน
controllers/	
auth.controller.js	- ตรวจสอบรหัสผ่าน (Bcrypt), สร้าง JWT Token และจัดการสิทธิ์ผู้ใช้
project.controller.js	- คำนวณความก้าวหน้าโครงการ ระบบล็อกแผนงาน และตรวจสอบสิทธิ์ IDOR
activity.controller.js	- บันทึกผลการดำเนินงานกิจกรรม คำนวณยอดเงินสะสม และจัดการรูปภาพหลักฐาน
dashboard.controller.js	- ประมวลผลสถิติ กรองโครงการติดธงแดง (Red Flags) ตามช้อยกเว้น MBE
master.controller.js	- ตรวจจับการเพิ่ม/ลบ/แก้ไขข้อมูลยุทธศาสตร์ 4 ระดับ และหน่วยงาน
issue.controller.js	- รับเรื่องแจ้งปัญหา บันทึกรายละเอียด และเปลี่ยนสถานะโดยผู้ดูแลระบบ
report.controller.js	- คำนวณยอดรวมงบประมาณและผลผลิตตามยุทธศาสตร์เพื่อออกรายงาน
prisma/	
schema.prisma	- แบบจำลองฐานข้อมูล 14 ตาราง Enums ความสัมพันธ์ และกฎความปลอดภัย
seed.js	- สคริปต์หลักสำหรับสร้าง Master Data และบัญชีทดสอบครบทุก Role
migrations/	- โฟลเดอร์เก็บประวัติและไฟล์สคริปต์การอัปเดตโครงสร้างฐานข้อมูล
uploads/	- โฟลเดอร์จัดเก็บไฟล์รูปภาพหลักฐานกิจกรรมที่อัปโหลดขึ้นเซิร์ฟเวอร์
frontend/	(ระบบหน้าบ้าน React 19 + Vite + Tailwind CSS)
index.html	- โครงสร้าง HTML หลัก จุดติดตั้ง Root Element ของ React
vite.config.js	- กำหนดค่า Vite, ตั้งค่า Proxy เชื่อมต่อไปยัง Backend และพอร์ต
tailwind.config.js	- กำหนดคัลสีหลัก (primary: #6C3BFF) ฟอนต์ และ Utility คลาสของระบบ
postcss.config.js	- ตั้งค่า PostCSS สำหรับคอมไพล์สไตล์ร่วมกับ Tailwind และ Autoprefixer
vercel.json	- กฎ Rewrite ส่งทุก URL ไปที่ index.html ป้องกันหน้า 404 บน Vercel
package.json / package-lock.json	- กำหนด Dependencies และคำสั่งสคริปต์รันของ Frontend
src/	(โค้ดต้นฉบับฝั่งหน้าบ้าน)
main.jsx	- จุดเริ่มต้น React นำ App ไปเรนเดอร์ใน DOM และห่อหุ้มด้วย Router
App.jsx	- จัดการเส้นทาง URL (Routing) และระบบป้องกันสิทธิ์ (Role Guard)
index.css	- ไฟล์สไตล์ CSS หลัก และการตั้งค่าพื้นฐานของ Tailwind CSS
layouts/	
AppLayout.jsx	- Master Layout รวม Sidebar + Topbar + Outlet (พื้นที่แสดงเนื้อหา)
components/	
Topbar.jsx	- แถบเมนูด้านบน แสดงข้อมูลระบบ แจ้งเตือน และโปรไฟล์ผู้ใช้
Sidebar.jsx	- เมนูแถบข้างด้านซ้าย ปรับเปลี่ยนรายการตาม Role ของผู้ใช้โดยอัตโนมัติ
CustomSelect.jsx	- Dropdown พิเศษที่รองรับการค้นหาตัวเลือก (Searchable Dropdown)
ExecutiveProjectModal.jsx	- หน้าต่าง Pop-up สรุปข้อมูลโครงการสำหรับผู้บริหาร
ProfileModal.jsx	- หน้าต่าง Pop-up แสดงข้อมูลผู้ใช้และฟอร์มเปลี่ยนรหัสผ่าน
ReportIssueModal.jsx	- หน้าต่าง Pop-up สำหรับกรอกและส่งเรื่องแจ้งปัญหาไปยังแอดมิน
ErrorBoundary.jsx	- ตัวดักจับข้อผิดพลาดของ UI เพื่อป้องกันไม่ให้หน้าเว็บล่มทั้งระบบ
contexts/	

└─ AuthContext.jsx	– จัดการ State การล็อกอิน ข้อมูลผู้ใช้ สิทธิ์ และฟังก์ชัน Login/Logout
└─ services/	
└─ api.js	– กำหนดค่า Axios Client และตั้งค่า JWT Token ในทุกคำขออัตโนมัติ
└─ utils/	
└─ imageUrl.js	– แปลง Path สัมผัสของรูปภาพให้เป็น Full URL สำหรับแสดงผลบนหน้าเว็บ
└─ statusHelper.js	– ฟังก์ชันแปลงรหัสสถานะโครงการ/กิจกรรมเป็นสี Badge และข้อความภาษาไทย
└─ pages/	
└─ auth/Login.jsx	– หน้าจอกรอกชื่อผู้ใช้และรหัสผ่านเพื่อเข้าสู่ระบบ
└─ teacher/	– แดชบอร์ดอาจารย์, จัดการโครงการ, ฟอรัม 4 ระดับ, บันทึกกิจกรรม, แกลเลอรี
└─ dean/	– แดชบอร์ดคณะ, โครงการคณะ, ตารางเปรียบเทียบสาขาวิชา, ส่งการคณบดี
└─ president/	– แดชบอร์ดสถาบัน, ตรวจจับโครงการติดธงแดง, Strategic Heatmap
└─ admin/	– แดชบอร์ดระบบ, จัดการ Master Data 9 หมวด, ปลดล็อกโครงการ, จัดการปัญหา
└─ executive/	– หน้ารายละเอียดโครงการแบบสรุปสำหรับผู้บริหาร